

Evidence-Aware Hallucination Reduction for 3GPP RAN

Abhay Shashidhara, Chaitanya Kadam, Francisco Sevilla
shash024@umn.edu, kadam032@umn.edu, sevil028@umn.edu
University of Minnesota Twin Cities

Abstract

Large Language Models have become useful for answering technical questions, but they can still generate unsupported or hallucinated answers when used with dense technical specifications. This project focuses on reducing hallucination in a 3GPP RAN question-answering system by grounding answers in retrieved evidence from 3GPP TS 38.331. The system retrieves relevant specification text, generates an answer using an instruction-tuned language model, and then verifies the answer using a Natural Language Inference based judge. If the answer is not sufficiently supported, the system either performs adaptive retrieval or abstains with “Not enough information.” The goal is to improve reliability for technical question answering in telecom specifications where unsupported answers can be misleading.

1 Introduction

Wireless network technology is standardized by 3GPP, the 3rd Generation Partnership Project. The 5G cellular network ecosystem depends on a large collection of technical specifications that define how different network components should communicate and operate. These specifications describe protocols used by base stations, user equipment, and other parts of the cellular network.

Among these documents, the TS 38 series contains important information related to 5G New Radio. In this project, the main focus is on TS 38.331, which describes Radio Resource Control, or RRC, behavior. RRC is a key protocol because it handles important control-plane procedures between the user equipment and the radio access network.

However, 3GPP technical specifications are long, dense, and difficult to read manually. They contain detailed procedures, message definitions, configuration fields, timers, conditions, and protocol states. Even experts usually learn these documents gradually through repeated use. For students, researchers, and engineers who are new to the domain, finding the correct information from the specification can be time-consuming.

This project aims to build a question-answering system for 3GPP RAN and RRC-related questions. The goal is



Figure 1: 5G network technology is the main technical domain used in this project.

not only to generate fluent answers, but also to make sure that the answers are supported by evidence from the specification. This is important because a normal language model may produce a confident answer even when the answer is not grounded in the technical document.

2 Background

AI chatbots are all over the world leveraging LLMs, or Large Language Models, because these models can produce human-like interactions in natural language and conversation style. When dealing with technical documents, LLMs tend to hallucinate unless they are grounded in the provided context. A hallucination occurs when a model generates content that sounds correct but is not actually supported by the source material. In a technical domain such as 3GPP RAN, this can be a serious issue. For example, an unsupported answer may mention an incorrect message name, timer, procedure, state transition, or configuration condition. Even if the answer sounds fluent, it can still be technically wrong.

Retrieval-Augmented Generation, or RAG, is one way to reduce this problem. In a RAG system, the model does not answer only from its internal knowledge. Instead, the system first retrieves relevant text from a trusted source and then asks the model to answer using that retrieved context. For this project, the trusted primary source is 3GPP TS 38.331. However, retrieval alone does not fully solve hallucination. Even when relevant chunks are retrieved, the generator can still add unsupported details or misinterpret the evidence. Because of this, the project adds an evidence-aware verification step. After the an-

swer is generated, a Natural Language Inference based judge checks whether the answer is supported by the retrieved evidence.

This makes the system more cautious than a standard RAG pipeline. If the evidence is strong, the answer can be returned. If the evidence is weak or the judge finds that the answer is not supported, the system can either perform adaptive retrieval or return “Not enough information.” This design is especially useful for technical question answering because it prioritizes correctness and grounding over simply producing an answer.

3 Methods

The main goal of this project was to build a question-answering system for 3GPP RAN technical questions that does not rely only on the language model’s internal knowledge. Instead, the system first searches for relevant evidence, generates an answer from that evidence, and then checks whether the generated answer is actually supported. This design was chosen because 3GPP specifications are highly technical, and even a fluent answer can be unsafe if it introduces unsupported details such as incorrect timers, message names, procedures, or configuration conditions.

3.1 Pipeline Overview

The overall pipeline begins with a user question about 3GPP RAN behavior, procedures, messages, or configuration details. The system first treats 3GPP TS 38.331 as the primary source because the project focuses specifically on RRC-related information. The PDF text is extracted, cleaned, and divided into overlapping chunks so that the retriever can search smaller pieces of the specification instead of processing the entire document at once. After retrieving candidate chunks, the system selects evidence sentences from those chunks using lexical overlap with the question. This step helps reduce the amount of unrelated context passed to the generator and keeps the answer focused on the most relevant parts of the retrieved specification text.

The selected evidence is then passed to the generator model with a strict instruction prompt. The prompt tells the model to answer only from the provided evidence, avoid outside knowledge, and abstain with “Not enough information” if the evidence does not directly support the answer. This is important because the project is not only trying to produce correct answers, but also trying to avoid unsupported answers in cases where the retrieved

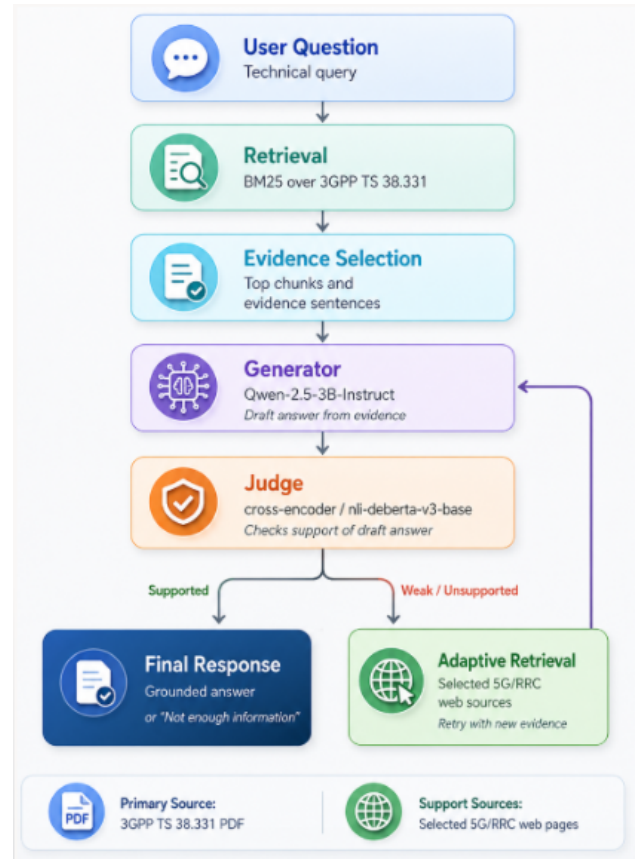


Figure 2: Overall evidence-aware RAG pipeline used for 3GPP RAN question answering.

information is weak. After the draft answer is generated, it is not immediately returned to the user. Instead, the answer is passed to a verification stage using an NLI-based judge. The judge compares the generated claims against the selected evidence and decides whether the answer is supported, partially supported, unsupported, or should be rejected. If the answer is supported, the system returns it as the final response. If the answer is weak or unsupported, the system either moves to adaptive retrieval or returns “Not enough information.”

3.2 Text Extraction

The first implementation step was to extract usable content from the 3GPP TS 38.331 PDF. Since this document is a long technical specification, it contains dense page-level text, section numbers, definitions, procedures, information element descriptions, and some structured content such as tables. Before retrieval could be done, this content had to be converted into a machine-readable format.

For this step, we used a Python extraction script with multiple PDF-processing libraries. The script first tries

pdfplumber, because it can extract both regular text and tables from each page. If that method fails, it uses **PyMuPDF**, which is fast and reliable for extracting page text and basic layout information. As a final fallback, the script uses **PyPDF2** to extract text from the PDF. Using multiple libraries made the extraction step more robust because technical PDFs do not always have simple formatting. For each page, the script extracts the available text and stores it with the page number. When tables are detected, they are also extracted and saved along with the page content. The script also records basic metadata such as the number of pages and PDF metadata. The extracted output is saved in two formats: a structured JSON file and a plain text file. The JSON version keeps page-level information in an organized format, while the text file makes it easier to manually inspect whether the extraction worked correctly.

3.3 Primary Retrieval and Evidence Selection

The primary retrieval stage starts with a technical 3GPP RAN question and searches 3GPP TS 38.331 using BM25. BM25 is a keyword-based retriever that ranks document chunks based on how strongly their terms match the user’s question. The system uses **prefetch.k = 120** to first collect a wider set of candidate chunks. From this larger pool, it keeps the final **top.k = 8** most relevant chunks. These chunks are then used for evidence selection. Instead of passing the full retrieved chunks directly to the generator, the system selects up to 18 evidence sentences from them. The selection uses a minimum overlap value of **0.01**, so only sentences that have some connection to the question are kept.

This step makes the evidence more focused before it is passed to the generator. It also reduces unrelated context from the specification, which helps the later answer generation and judging stages stay grounded.

3.4 Answer Generation

After evidence selection, the selected evidence sentences are passed to the answer generation model. In this project, we used **Qwen-2.5-1.5B-Instruct** as the generator. This is an instruction-tuned language model, so it can follow a strict prompt and produce a structured technical answer from the retrieved evidence. The generator receives the user question, the selected evidence sentences, and an instruction prompt. The prompt tells the model to answer only from the provided evidence and not from general knowledge. It also tells the model to avoid adding unsupported details such as message names, timers, states,

Table 1: Primary judge decision thresholds.

Decision	Judge 1: Primary
Supported	Entailment ≥ 0.50 or support score ≥ 0.34
Partially supported	$0.25 \leq \text{entailment} < 0.50$
Unsupported / Reject	Contradiction > 0.75 or below partial threshold

procedures, causes, or configuration fields.

The generation step uses **max_new_tokens = 450**, which limits the maximum length of the draft answer. This gives the model enough space to explain the answer clearly while preventing it from producing an unnecessarily long response. The model is also run with **temperature = 0.0**, which makes the output more stable and less random. Since the goal is evidence-grounded technical answering, this setting helps keep the generated answer consistent. At this stage, the generated response is treated only as a draft answer. It is not returned directly to the user until it is checked by the NLI judge in the next step.

3.5 Judge Implementation

After the generator creates a draft answer, the system uses a Natural Language Inference, or NLI, judge to check whether the answer is supported by the selected evidence. Natural Language Inference compares two pieces of text and checks whether the generated answer follows from the evidence or conflicts with it. The judge used in this project is **cross-encoder/nli-deberta-v3-base**. The generated answer is compared against the retrieved evidence, and the judge produces scores such as entailment, contradiction, and support score. Entailment shows how strongly the evidence supports the answer, contradiction shows whether the answer conflicts with the evidence, and support score captures whether enough of the generated answer is backed by the evidence.

The system uses two judge settings: one for the primary retrieval stage and one for the adaptive retrieval stage. The primary judge is stricter because the system first expects the answer to be grounded in the official TS 38.331 specification. The adaptive judge is slightly more flexible because it is used only as a fallback when the primary evidence is weak.

Based on these scores, the answer is classified as supported, partially supported, or unsupported/rejected. If the answer is supported, it can move forward as a grounded response. If it is partially supported, the system treats it cautiously. If it is unsupported or rejected, the system does not trust the draft answer.

Table 2: Adaptive judge decision thresholds.

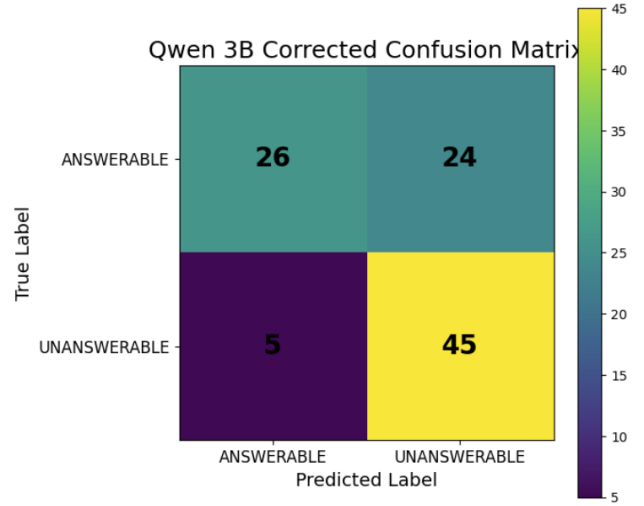
Decision	Judge 2: Adaptive
Supported	Entailment ≥ 0.35 or support score ≥ 0.20
Partially supported	$0.20 \leq \text{entailment} < 0.35$
Unsupported / Reject	Contradiction > 0.80 or below partial threshold

3.6 Adaptive Retrieval

After the judge evaluates the primary draft answer, the system decides whether the evidence from 3GPP TS 38.331 is strong enough. If the primary answer is supported, the system returns it. If the primary evidence is weak or the draft answer is not well supported, the pipeline moves to adaptive retrieval. Adaptive retrieval is implemented as a controlled fallback over 10 embedded URLs, not as an open web search. This means the system has a second evidence source available, but it is still limited to preselected sources.

When the adaptive retriever is built, each URL is fetched and converted into text. If the source is a PDF, the system extracts text from its pages and combines it. If the source is an HTML page, the system uses **BeautifulSoup** to remove noisy page elements such as scripts, styles, navigation bars, footers, and other non-content tags. The remaining page text is normalized before chunking. The cleaned adaptive text is then split into smaller chunks of 360 words with a 60-word overlap. Chunks that are too short or look like generic noise are skipped. Each valid adaptive chunk is stored with its source title and URL, and then a separate BM25 retriever is created over all adaptive chunks.

When adaptive retrieval is triggered, the original user question is searched again over this adaptive BM25 index. The system keeps the top adaptive results using **adaptive_top_k = 8**, then performs the same evidence sentence selection step as before. The generator uses this adaptive evidence to create a new draft answer, and the NLI judge checks it again before the final response is returned. If the adaptive answer is supported, it becomes the final answer. If the adaptive evidence is still weak or the judge rejects the draft, the system returns “Not enough information.” This makes adaptive retrieval a second chance to find better evidence, not a shortcut for unsupported answering.

**Figure 3:** Confusion matrix for the Qwen2.5-3B evidence-aware RAG system on 100 evaluation questions.

4 Experiment Results

We evaluated the proposed evidence-aware RAG system on a balanced test set of 100 questions, consisting of 50 answerable 3GPP RAN/RRC questions and 50 unanswerable or out-of-domain questions. The goal of this evaluation was not only to measure whether the system could generate correct answers, but also whether it could correctly decide when to abstain and return “Not enough information.” The main Qwen2.5-3B system achieved a corrected token F1 of 0.485, answerability F1 of 0.642, unanswerable rejection rate of 0.900, hallucination rate of 0.100, and evidence support rate of 1.000. These results show that the system was highly effective at rejecting unsupported questions while still producing evidence-backed answers for a subset of answerable questions.

As shown in Fig. 3, Qwen2.5-3B correctly answered 26 out of 50 answerable questions and correctly rejected 45 out of 50 unanswerable questions. Only 5 out of 50 unanswerable questions were incorrectly answered, giving a hallucination rate of 10%. However, the system also abstained on 24 out of 50 answerable questions, showing that the model is conservative.

4.1 Results and Analysis

The results show a clear trade-off between hallucination reduction and answer coverage. The Qwen2.5-3B system performed strongly on safety-oriented metrics. It rejected 90% of unanswerable questions and achieved an evidence support rate of 100%, meaning every final answered output was backed by retrieved evidence. This supports the main goal of the project: reducing unsupported answers

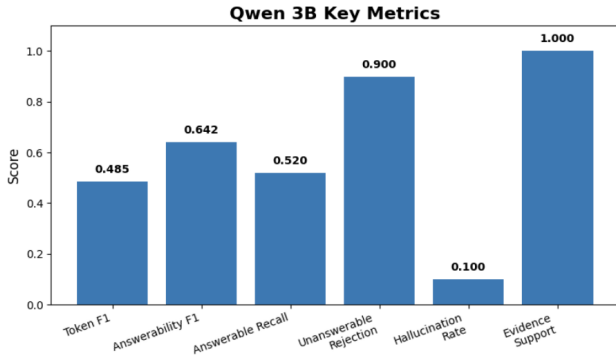


Figure 4: Key evaluation metrics for the Qwen2.5-3B evidence-aware RAG system.

in a technical 3GPP QA setting.

However, the system’s answerable recall was 0.520, meaning it answered only 52% of the answerable 3GPP questions. This indicates that the pipeline is conservative. The NLI judge and evidence filtering steps reduce hallucination, but they can also reject valid answers when the retrieved evidence is incomplete, phrased differently from the reference answer, or difficult for the verifier to classify as supported. The confusion matrix in Fig. 3 highlights this behavior. The model is much stronger at rejecting unanswerable questions than at answering all valid questions. This is useful for hallucination reduction, but it also means that future improvements should focus on improving answer coverage without increasing hallucination.

The key metrics in Fig. 4 further support this interpretation. The system achieved a high unanswerable rejection rate of 0.900 and perfect evidence support rate of 1.000, but a moderate token F1 of 0.485 and answerability F1 of 0.642. The low hallucination rate of 0.100 is one of the strongest outcomes of the system.

4.2 Model Evaluation

We compared three generator sizes under the same retrieval and NLI-based verification framework: Qwen2.5-0.5B, Qwen2.5-1.5B, and Qwen2.5-3B. The comparison is shown in Fig. 5.

Qwen2.5-0.5B had the highest answerable recall of 0.667, meaning it answered more answerable questions. However, it also had the highest hallucination rate of 0.500 and the lowest unanswerable rejection rate of 0.500. This suggests that the smaller model was less reliable at safely refusing unsupported questions. In some cases, it also produced unstable or placeholder-like outputs, making it less suitable for the final system.

Qwen2.5-1.5B improved safety compared with 0.5B,

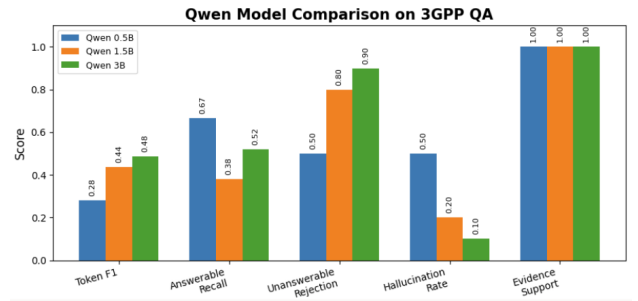


Figure 5: Comparison of Qwen2.5 generator sizes under the same retrieval and NLI-based verification framework.

reducing hallucination rate to 0.200 and increasing unanswerable rejection rate to 0.800. However, it had the lowest answerable recall of 0.380, which means it was too conservative and failed to answer many valid questions. Qwen2.5-3B achieved the best overall safety-performance trade-off. It had the best token F1 of 0.485, the highest unanswerable rejection rate of 0.900, and the lowest hallucination rate of 0.100. Although its answerable recall of 0.520 was lower than the 0.5B model, it was much safer and more reliable for a hallucination-reduction system. Therefore, Qwen2.5-3B was selected as the main generator for the final evidence-aware RAG pipeline.

4.3 Report Analysis

The experimental results support the main claim that evidence-aware verification can reduce hallucination in domain-specific technical QA. Compared with smaller generator models, Qwen2.5-3B showed stronger grounding behavior and better safety on unanswerable questions. The system’s use of BM25 retrieval, evidence extraction, Qwen generation, and DeBERTa-based NLI verification helped ensure that final answers were supported by evidence.

The strongest result is the low hallucination rate of 0.100 for Qwen2.5-3B. This means the system incorrectly answered only 5 out of 50 unanswerable questions. For a technical domain such as 3GPP RAN, this is important because unsupported answers can mislead users and cause incorrect interpretation of protocol behavior. At the same time, the results reveal an important limitation: the system is conservative. It abstains on many answerable questions, as shown by the answerable recall of 0.520. This suggests that the current verifier thresholds and retrieval pipeline prioritize safety over coverage. For this project, that is an acceptable design choice because the primary goal is hallucination reduction, but future work should aim to improve recall while maintaining a low hallucination rate.

Overall, the Qwen2.5-3B system demonstrates that combining retrieval with NLI-based judging is a promising direction for evidence-aware QA over technical specifications. The model does not simply generate answers; it attempts to verify whether the answer is supported by retrieved evidence. This makes the system more reliable than a standard RAG pipeline, especially for high-precision domains such as telecom specifications.

5 Discussion Points

5.1 Replicability

Our system is designed to be reasonably reproducible because the pipeline follows a clear sequence: document extraction, chunking, BM25 retrieval, answer generation, NLI-based judging, and adaptive retrieval when needed. We used a single version of 3GPP TS 38.331 as the primary document to avoid conflicts between different specification releases. This makes the retrieval source stable and reduces ambiguity in the answers.

The main reproducibility factors are the chunking parameters, retrieval settings, model choices, and judge thresholds. For example, the system uses fixed chunk sizes, overlap values, top-k retrieval settings, and NLI thresholds. If another user has access to the same PDF, the same extracted text, and the same model versions, they should be able to reproduce similar outputs. However, exact reproducibility may still vary slightly because local GPU hardware, transformer library versions, tokenizer behavior, and model loading settings can affect generation speed and, in some cases, output wording. To improve replicability, all configuration values, model names, prompts, and evaluation questions should be documented clearly.

5.2 Datasets

The dataset used in this project mainly consists of the extracted text from 3GPP TS 38.331 and a small evaluation set of 3GPP/RRC-related questions. Since the primary goal is evidence-aware question answering, the quality of the extracted text and the relevance of the test questions directly affect system performance. If the PDF extraction introduces formatting errors, broken sentences, or noisy table-of-contents chunks, retrieval quality can decrease and the model may answer from the wrong evidence.

The question set also affects the direction of the project. If the questions are mostly narrow technical questions, the system may appear strong because BM25 retrieval can locate exact terms such as SIB1, RRCReconfigura-

tion, or ul-GapFR2-Config. If the questions are broader, such as “What are general UE requirements?”, the system may struggle because the relevant evidence may be spread across many sections. Therefore, the dataset and question design influence whether future work should focus more on retrieval improvement, semantic search, query expansion, or better evaluation design.

5.3 Ethics

This work has potential societal and technical risks because it deals with telecommunications specifications, which are used in real-world wireless communication systems. A hallucinated or unsupported answer could mislead engineers, researchers, or students who rely on the system for understanding 3GPP procedures. In a safety-critical or production environment, incorrect interpretation of RRC procedures could lead to implementation mistakes, interoperability issues, or incorrect assumptions about network behavior.

To reduce this risk, our system is designed to answer only when the retrieved evidence supports the response. The NLI judge checks whether the generated answer is grounded in the selected evidence, and the system can output “Not enough information” or mark a question as out of context when support is weak. Another ethical concern is over-trust: users may assume the system is always correct because it cites technical evidence. To address this, the system should display retrieved evidence, route information, confidence or judge scores, and clear limitations. This makes the model more transparent and encourages users to verify important answers against the original specification.

5.4 Limitations

- **Over-abstention:** The system reduces hallucination well, but it sometimes rejects valid answerable 3GPP questions.
- **BM25 retrieval dependency:** BM25 relies on keyword overlap, so it may miss relevant evidence when the question is phrased differently from the specification.
- **NLI judge limitation:** The judge checks answer-evidence support, but it may not always verify whether the evidence fully answers the original question.
- **Adaptive source noise:** Adaptive retrieval helps recover failed cases, but external sources can introduce noisy or less precise evidence.

- **Small evaluation set:** The 50 answerable and 50 unanswerable questions are useful for testing, but they do not cover the full complexity of real telecom QA.
- **Generic unanswerable questions:** Some unanswerable questions are broad out-of-domain examples; future work should include harder telecom-specific adversarial questions.
- **Metric limitation:** Token F1 and exact match may underestimate quality when the generated answer is correct but phrased differently from the reference answer.

5.5 Future Work

Future work can extend this project in several directions. First, semantic retrieval using embeddings or hybrid BM25 plus dense retrieval could improve performance on broad and paraphrased questions. Second, a reranker could be added after retrieval to better select the most relevant chunks before generation. Third, the evaluation set can be expanded with more answerable and unanswerable questions, including expert-written reference answers. Fourth, the system can be extended to multiple 3GPP specifications while carefully tracking version metadata to avoid conflicts. Finally, the frontend can be improved to show evidence snippets, judge scores, response time, and route information more clearly so users can understand why an answer was accepted or rejected.

6 Contribution

Each team member contributed to a major part of the project, and the final system was completed through a shared effort across design, implementation, evaluation, and report writing.

Francisco Sevilla contributed to the initial idea pitch, data collection, and text processing. His work helped define the project direction and supported the preparation of the source material needed for retrieval and evaluation.

Abhay Shashidhara contributed to the overall pipeline design, hyperparameter tuning, generator implementation, adaptive retrieval. His work focused on connecting the retrieval, generation, and verification stages into a complete evidence-aware RAG pipeline.

Chaitanya Kadam contributed to the retriever implementation, judge implementation, and model evaluation. His work helped strengthen the retrieval and verification

components and supported the comparison of different generator models.

Overall, the project was a collaborative effort where each member contributed to a different but equally important part of the final system and report writing.

7 References

1. Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 2020.
2. Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. Enabling Large Language Models to Generate Text with Citations. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.
3. Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. *International Conference on Learning Representations*, 2024.
4. Stephen Robertson and Hugo Zaragoza. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 2009.
5. Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense Passage Retrieval for Open-Domain Question Answering. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 2020.
6. Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. DeBERTa: Decoding-enhanced BERT with Disentangled Attention. *International Conference on Learning Representations*, 2021.
7. Shehzaad Dhuliawala, Mojtaba Komeili, Jing Xu, Roberta Raileanu, Xian Li, Asli Celikyilmaz, and Jason Weston. Chain-of-Verification Reduces Hallucination in Large Language Models. *Findings of the Association for Computational Linguistics: ACL*, 2024.

8. Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 2023.
9. Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego de Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, Tom Hennigan, Saffron Huang, Loren Maggiore, Chris Jones, Albin Cassirer, Andy Brock, Michela Paganini, Geoffrey Irving, Oriol Vinyals, Simon Osindero, Karen Simonyan, Jack Rae, Erich Elsen, and Laurent Sifre. Improving Language Models by Retrieving from Trillions of Tokens. *International Conference on Machine Learning*, 2022.
10. Nelson F. Liu, Tianyi Zhang, and Percy Liang. Evaluating Verifiability in Generative Search Engines. *Findings of the Association for Computational Linguistics: EMNLP*, 2023.
9. 5G NR overview: https://en.wikipedia.org/wiki/5G_NR
10. 3GPP 5G system overview: <https://www.3gpp.org/technologies/5g-system-overview>

The adaptive retrieval stage used the following 10 preselected URLs as controlled fallback sources:

1. ETSI TS 138331 V18.1.0 PDF: https://www.etsi.org/deliver/etsi_ts/138300_138399/138331/18.01.00_60/ts_138331v180100p.pdf
2. Radio Resource Control overview: https://en.wikipedia.org/wiki/Radio_Resource_Control
3. 5G RRC protocol overview: <https://www.3gppinfo.com/5g/protocols/rrc/>
4. 5G NR RRC overview: <https://www.telcomaglobal.com/p/5g-nr-rrc>
5. LTE-NR interworking in 5G: <https://www.ericsson.com/en/blog/2019/5/lte-nr-interworking-in-5g>
6. EventHelix 5G reference: <https://www.eventhelix.com/5G/>
7. Qualcomm 5G NR research overview: <https://www.qualcomm.com/research/5g/5g-nr>
8. Techplayon 5G NR overview: <https://www.techplayon.com/5gnr/>